Introducción

Como estudiante de programación, aprender a utilizar un entorno de desarrollo integrado (IDE) eficiente es crucial para el éxito en tus cursos de Python, C, C++ y Java. Visual Studio Code (VSCode) es una opción excepcional para estudiantes de todos los niveles, ya que combina facilidad de uso, personalización y un amplio soporte para lenguajes de programación. Hemos reconocido otros IDE como JetBrains CLion, IntelliJ IDEA y Eclipse, en este curso nos centraremos en el uso de VSCode para sus estudios de programación en Python, C, C++ y Java.

Elegir Visual Studio Code

Visual Studio Code (VSCode) es un entorno de desarrollo integrado (IDE) ligero, gratuito y de código abierto creado por Microsoft. Es compatible con una amplia variedad de lenguajes de programación, como Python, C, C++ y Java, lo que lo convierte en una opción versátil y accesible para estudiantes de programación. Además, VSCode funciona en múltiples sistemas operativos, incluidos Windows, macOS y Linux, lo que garantiza su compatibilidad con la mayoría de las computadoras.

Una de las principales ventajas de VSCode es su capacidad para personalizarse mediante extensiones y complementos. Esto les permite a los estudiantes adaptar el entorno de desarrollo a sus necesidades específicas, lo que puede facilitar el aprendizaje y mejorar la eficiencia en el trabajo. La comunidad de usuarios y desarrolladores de VSCode es activa y está en constante creación y mejora de extensiones.

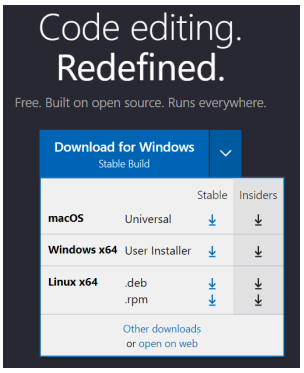
Para obtener más información sobre Visual Studio Code, visiten su sitio web oficial:

https://code.visualstudio.com

En este curso, exploraremos las características y funcionalidades de VSCode para que puedan sacarle el máximo provecho en sus estudios de programación en Python, C, C++ y Java.

Instalación de Visual Studio Code

La instalación de Visual Studio Code es un proceso sencillo y fácil de seguir. A continuación, se presentan los pasos detallados para instalar VSCode en tu sistema operativo.



Paso 1: Descargar Visual Studio Code

Diríjase al sitio web oficial de Visual Studio Code en https://code.visualstudio.com/.

En la página principal, encontrará botones de descarga para Windows, macOS y Linux. De igual manera Visual Studio Code detectará automáticamente tu sistema operativo y te ofrecerá la versión adecuada. Haz clic en el botón de descarga para comenzar a descargar el instalador.

Paso 2: Ejecutar el instalador

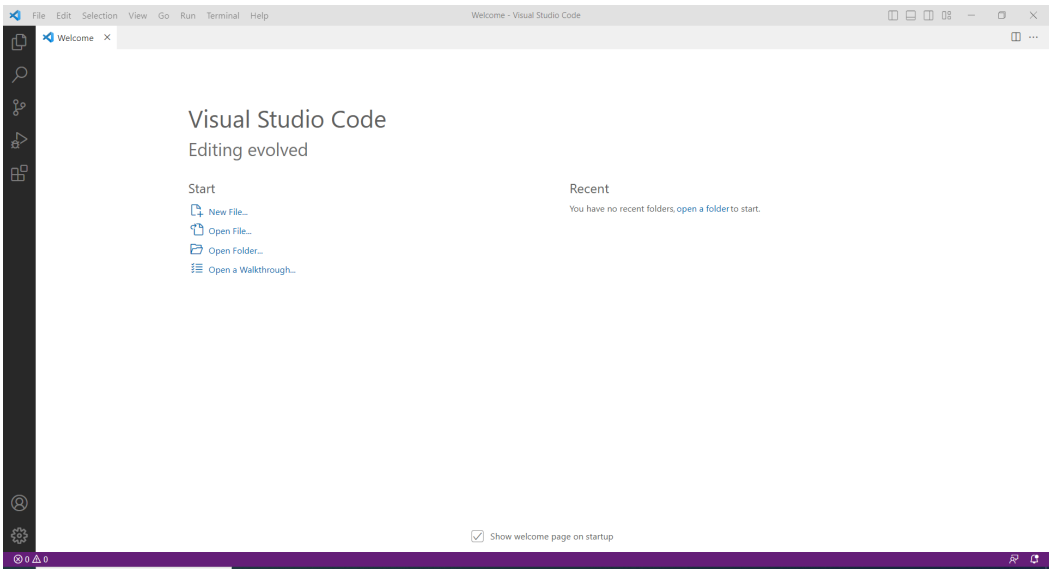
Una vez que haya descargado el archivo de instalación, localízalo en su computadora y haga doble clic para ejecutarlo.

En Windows: Ejecute el archivo .exe descargado y siga las instrucciones del instalador. Acepte el acuerdo de licencia y elija la carpeta de instalación. También puede seleccionar opciones adicionales, como crear un acceso directo en el escritorio o agregar la opción "Abrir con Code" al menú contextual. Finalmente, haga clic en "Instalar" para iniciar la instalación.

En macOS: Abra el archivo .zip descargado y arrastre el icono de Visual Studio Code a la carpeta "Aplicaciones". Luego, abra la carpeta "Aplicaciones" y haga clic en el icono de Visual Studio Code para iniciarlo por primera vez. macOS verificará la aplicación y la abrirá.

En Linux: Extraiga el archivo .tar.gz descargado y ejecute el script "code" dentro de la carpeta extraída. Dependiendo de su distribución de Linux, también puede usar un administrador de paquetes para instalar Visual Studio Code. Consulte las instrucciones específicas para su distribución en la documentación oficial de Visual Studio Code.

Paso 3: Verificar la instalación
Una vez completada la instalación, abra Visual Studio Code desde el acceso directo creado en su escritorio, el menú de aplicaciones o la terminal. Debería ver la ventana principal de Visual Studio Code con una barra lateral, un área de edición y una barra de estado en la parte inferior.



¡Felicidades! Ahora ha instalado correctamente Visual Studio Code en su computadora. A continuación, configuraremos el entorno de desarrollo para cada uno de los lenguajes de programación: C, C++ y Java.

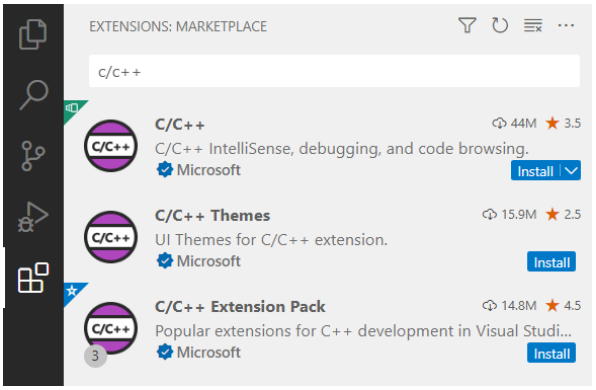
Configuración de entornos de desarrollo para cada lenguaje

4.1. Configuración para C

En esta sección, aprenderemos cómo configurar el entorno de desarrollo de Visual Studio Code para el lenguaje de programación C. Seguiremos los pasos para instalar las extensiones necesarias, configurar el compilador y personalizar el entorno según nuestras preferencias.

Paso 1: Instalar las extensiones necesarias

Visual Studio Code admite una gran cantidad de extensiones para mejorar y personalizar la experiencia del usuario. Para trabajar con el lenguaje C, necesitamos instalar algunas extensiones específicas.



Abra Visual Studio Code y haga clic en el icono de extensiones en la barra lateral izquierda (o presione Ctrl+Shift+X).
Busque "C/C++" en el cuadro de búsqueda y seleccione la extensión "C/C++" desarrollada por Microsoft. Haga clic en el botón "Instalar" para instalar la extensión.

Esta extensión proporciona características esenciales para el desarrollo en C, como la resaltado de sintaxis, la navegación por el código y el soporte para la depuración.

Paso 2: Instalar el compilador GCC

Antes de comenzar a escribir código en C, es necesario instalar un compilador que convierta el código fuente en un archivo ejecutable. Uno de los compiladores más populares y ampliamente utilizados para C es GCC (GNU Compiler Collection). Para instalar GCC, sigue las instrucciones correspondientes a tu sistema operativo:

Windows: Descarga e instala MSYS2, que incluye GCC y otras herramientas esenciales para la programación en C y C++. Visita la página oficial de MSYS2 (https://www.msys2.org/) para obtener más información y seguir las instrucciones de instalación.

macOS: Instala Xcode Command Line Tools, que incluye GCC y otras herramientas de desarrollo. Abre la Terminal y ejecuta el siguiente comando: xcode-select --install. Sigue las instrucciones en pantalla para completar la instalación.

Linux: La mayoría de las distribuciones de Linux ya incluyen GCC por defecto. Sin embargo, si no está instalado, abre la Terminal y utiliza el administrador de paquetes de tu distribución para instalarlo. Por ejemplo, en Ubuntu, ejecuta sudo apt-get install build-essential.

Seguimos ahora a configurar la variable PATH en Windows:
a. Si has instalado MSYS2, localiza el directorio de instalación (por defecto, se encuentra en C:\msys64 para la versión de 64 bits o C:\msys32 para la versión de 32 bits).

b. Dentro del directorio de MSYS2, busca la carpeta "mingw64\bin" (para la versión de 64 bits) o "mingw32\bin" (para la versión de 32 bits). Copia la ruta completa a esta carpeta.
c. Haz clic derecho en "Mi PC" o "Este equipo" en el Explorador de archivos y selecciona "Propiedades". Luego, haz clic en "Configuración avanzada del sistema" en el panel izquierdo.
d. En la pestaña "Opciones avanzadas", haz clic en "Variables de entorno".
e. En la sección "Variables del sistema", busca la variable "Path" y selecciónala. Luego, haz clic en "Editar".
f. En la ventana "Editar variable de entorno", haz clic en "Nuevo" y pega la ruta completa que copiaste en el paso b. Asegúrate de que no haya espacios ni caracteres adicionales antes o después de la ruta.
g. Haz clic en "Aceptar" para guardar los cambios y cerrar todas las ventanas abiertas.

Reinicia la consola de MSYS2 o abre una nueva ventana de la línea de comandos de Windows (cmd) y ejecuta el siguiente comando para verificar que GCC se instaló correctamente y se encuentra en la ruta PATH:

```
gcc --version
```

Si ves la versión de GCC, esto indica que la instalación y configuración fueron exitosas!!!

Paso 3: Configurar la extensión C/C++

a. Abre Visual Studio Code y haz clic en el icono de la rueda dentada en la esquina inferior izquierda de la ventana para abrir el menú de configuración.

b. Selecciona "Configuración" en el menú desplegable. Esto abrirá una nueva pestaña de configuración en el editor.

c. En el cuadro de búsqueda en la parte superior, escribe "C/C++" para filtrar las opciones de configuración relacionadas con la extensión C/C++.

d. Bajo la sección "Extensiones", haz clic en el enlace "Editar en settings.json" que se encuentra debajo de "Configuración de C/C++". Esto abrirá el archivo "settings.json" en el editor.

e. Aquí es donde puedes personalizar la configuración de la extensión C/C++ para que funcione con tu compilador y entorno específicos. Por ejemplo, puedes agregar las siguientes líneas de configuración para establecer el compilador y las opciones de depuración de GCC en Windows:

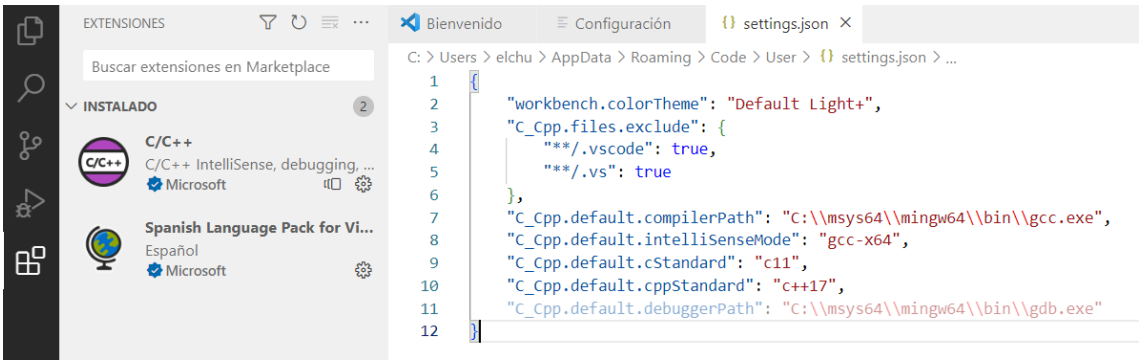
```
{
    "C_Cpp.default.compilerPath": "C:\\msys64\\mingw64\\bin\\gcc.exe",
    "C_Cpp.default.intelliSenseMode": "gcc-x64",
    "C_Cpp.default.cStandard": "c11",
    "C_Cpp.default.cppStandard": "c++17",
    "C_Cpp.default.debuggerPath": "C:\\msys64\\mingw64\\bin\\gdb.exe"
}
```

Asegúrate de ajustar las rutas según la ubicación en la que instalaste MSYS2 y MinGW. Si estás utilizando macOS o Linux, asegúrate de ajustar las rutas del compilador y del depurador según corresponda.

f. Guarda los cambios en el archivo "settings.json" y cierra la pestaña.

Ahora la extensión C/C++ debería estar configurada correctamente para funcionar con tu entorno de desarrollo en C. Puedes probar la configuración creando un nuevo archivo de código C y observando

si el resaltado de sintaxis, la finalización de código y otras características funcionan como se esperaba.



Paso 4: Crear y configurar un archivo tasks.json
El archivo tasks.json permite definir y configurar tareas en Visual Studio Code que se pueden ejecutar fácilmente desde el entorno de desarrollo. En este caso, configuraremos una tarea para compilar código C usando GCC.

a. En Visual Studio Code, ve a la barra de menú y selecciona "Terminal" > "Configurar tareas...".
b. Haz clic en "Crear tasks.json desde una plantilla" y elige "Otros" en la lista de plantillas disponibles.
c. Se creará un nuevo archivo tasks.json en la carpeta ".vscode" de tu espacio de trabajo. Reemplaza el contenido del archivo con el siguiente código:

```
{
    "version": "2.0.0",
    "tasks": [
       {
        "label": "build_c",
        "type": "shell",
        "command": "gcc",
        "args": [
          "-g",
          "*.c",
          "-o",
          "${fileDirname}\\main.exe"
        ],
        "options": {
            "cwd": "${fileDirname}"
        },
        "group": {
          "kind": "build",
          "isDefault": true
        },
        "problemMatcher": [
          "$gcc"
        ]
```

```
        }
    ]
}
```

d. Guarda el archivo tasks.json.

Este es un archivo de configuración de tareas para Visual Studio Code. Este archivo define una tarea llamada "build_c" que compila el código fuente C en el directorio actual y genera un archivo ejecutable "main.exe". La tarea se ejecuta mediante el comando "gcc" y pasa los argumentos "-g", "*.c", "-o" y "${fileDirname}\main.exe" al compilador.

El argumento "-g" indica al compilador que incluya información de depuración en el archivo ejecutable, lo que es útil para depurar el código. El argumento "*.c" indica que el compilador debe compilar todos los archivos fuente C en el directorio actual. El argumento "-o" especifica el nombre y la ubicación del archivo ejecutable generado. "${fileDirname}" es una variable de entorno de Visual Studio Code que se expande al directorio actual.

La opción "cwd" se establece en "${fileDirname}" para asegurarse de que la tarea se ejecute en el directorio actual. Esto es importante porque la tarea utiliza el comodín "*.c" para compilar todos los archivos fuente C en el directorio actual.

La tarea se agrupa como "build" y se establece como la tarea predeterminada. Además, se utiliza el problemaMatcher "$gcc" para mostrar errores y advertencias en el panel de problemas.

Una vez que hayas configurado el archivo tasks.json, podrás compilar tu código C utilizando el comando "Terminal" > "Ejecutar tarea de compilación" en la barra de menú, o presionando las teclas "Ctrl + Shift + B" (o "Cmd + Shift + B" en macOS).

Paso 5: Crear y configurar un archivo launch.json
El archivo launch.json es esencial para configurar el proceso de depuración en Visual Studio Code. Para crear y configurar un archivo launch.json para el lenguaje C, sigue estos pasos:

a. En Visual Studio Code, haz clic en el ícono de "Depurar" en la barra lateral izquierda o presiona Ctrl+Shift+D para abrir la vista de depuración.
b. En la parte superior de la vista de depuración, haz clic en el menú desplegable que dice "No Configurations" (Sin configuraciones) y selecciona "Add Configuration" (Agregar configuración). Esto creará automáticamente un archivo launch.json en la carpeta ".vscode" de tu proyecto.
c. Visual Studio Code abrirá el archivo launch.json recién creado. Reemplaza el contenido del archivo con el siguiente código de configuración básica:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "C: Launch",
            "type": "cppdbg",
            "request": "launch",
            "program": "${workspaceFolder}/nombre-del-ejecutable",
            "args": [],
```

```
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}",
            "environment": [],
            "externalConsole": true,
            "MIMode": "gdb",
            "miDebuggerPath": "ruta-del-gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ],
            "preLaunchTask": "nombre-de-la-tarea-de-compilacion"
        }
    ]
}
```

d. Reemplaza nombre-del-ejecutable con el nombre del archivo ejecutable que se genera al compilar tu programa en C. Por ejemplo, si tu archivo de código fuente se llama "main.c" y se compila en "main.exe", reemplaza nombre-del-ejecutable con "main.exe".
e. Reemplaza ruta-del-gdb con la ruta completa al ejecutable GDB que instalaste junto con GCC. Si estás utilizando MinGW en Windows, la ruta podría ser similar a "C:\msys64\mingw64\bin\gdb.exe". Asegúrate de usar barras invertidas dobles (\\) en la ruta.
f. Reemplaza nombre-de-la-tarea-de-compilacion con el nombre de la tarea de compilación que configuraste en el archivo tasks.json. Por ejemplo, si has definido una tarea llamada "build", utiliza "build" aquí.
g. Guarda el archivo launch.json.

Ahora tienes un archivo launch.json configurado para depurar programas en C en Visual Studio Code. Con esta configuración, podrás ejecutar y depurar tu código en C utilizando las herramientas de depuración integradas en Visual Studio Code.

Paso 6: Verificar la configuración y probar el entorno de desarrollo

Los archivos tasks.json y launch.json se deben crear en cada proyecto?

Sí, los archivos tasks.json y launch.json se deben crear en cada proyecto de Visual Studio Code donde quieras compilar y ejecutar código C, C++ o cualquier otro lenguaje compatible con las extensiones que estés utilizando. Estos archivos se almacenan en la carpeta .vscode dentro del directorio del proyecto.

El archivo tasks.json define las tareas de compilación y construcción específicas para el proyecto, lo que permite configurar el proceso de compilación de acuerdo con las necesidades de cada proyecto. Del mismo modo, el archivo launch.json define la configuración de depuración y ejecución para ese proyecto en particular.

Si vas a trabajar con varios proyectos que tienen configuraciones similares, puedes copiar y pegar los archivos tasks.json y launch.json de un proyecto a otro y hacer los ajustes necesarios. También puedes crear plantillas de estos archivos para facilitar la configuración en nuevos proyectos.

Al crear estos archivos en cada proyecto, aseguras que las configuraciones sean específicas para cada uno y no interfieran con otros proyectos en Visual Studio Code. Esto es especialmente útil cuando trabajas con diferentes lenguajes de programación o configuraciones de compilación y depuración en varios proyectos.

Ahora que has configurado los archivos tasks.json y launch.json en tu proyecto, es hora de verificar que todo esté funcionando correctamente. A continuación, se describen los pasos para probar tu entorno de desarrollo de C en Visual Studio Code:

a. Crear un archivo de código fuente en C: Si aún no lo has hecho, crea un nuevo archivo con extensión .c en tu proyecto y escribe un programa simple en C, como el siguiente "Hola, mundo":

```c
#include <stdio.h>

int main() {
    printf("Hola, mundo de Programacion!\n");
    return 0;
}
```

b. Guarda el archivo: Asegúrate de guardar el archivo de código fuente antes de continuar.
c. Compilar el programa: Para compilar el programa, abre la paleta de comandos de Visual Studio Code presionando Ctrl+Shift+P (o Cmd+Shift+P en macOS), escribe "Run Task" y selecciona "Run Build Task" (Ejecutar tarea de compilación). Elige la tarea de compilación que configuraste en el archivo tasks.json. Verifica que no haya errores en la salida del compilador.
d. Ejecutar y depurar el programa: Si la compilación fue exitosa, es hora de ejecutar y depurar el programa. Haz clic en el icono de "Depurar" en la barra lateral izquierda de Visual Studio Code y luego en el botón de "Iniciar Depuración" (el ícono de "play") en la parte superior de la ventana de depuración.

Si todo está configurado correctamente, Visual Studio Code ejecutará tu programa en modo de depuración, y podrás ver la salida del programa en la ventana de "Terminal" o "Depuración de la Consola".

Con estos pasos, has verificado que tu entorno de desarrollo de C en Visual Studio Code esté configurado correctamente y listo para compilar, ejecutar y depurar programas en C. Ahora puedes comenzar a desarrollar tus proyectos en C y aprovechar las herramientas y características de Visual Studio Code para mejorar tu productividad y la calidad del código.

4.2. Configuración para C++

Para configurar el entorno de desarrollo en Visual Studio Code para C++, sigue estos pasos:

Paso 1: Instalar las extensiones necesarias
Asegúrate de tener instalada la extensión C/C++ para Visual Studio Code, como se mencionó en la sección 4.1, Paso 1.

Paso 2: Instalar el compilador G++
Si ya instalaste el compilador GCC para la configuración de C, no es necesario realizar este paso, ya que G++ (compilador de C++) se incluye en el paquete de GCC. Si aún no lo has hecho, sigue las instrucciones de la sección 4.1, Paso 2.

Paso 3: Configurar la extensión C/C++
La configuración de la extensión C/C++ es la misma que en la sección 4.1, Paso 3. Asegúrate de que el estándar de C++ esté configurado correctamente (por ejemplo, "c++17").

Paso 4: Crear y configurar un archivo tasks.json
La creación y configuración del archivo tasks.json es similar a la sección 4.1, Paso 4, pero debes cambiar el comando de compilación a "g++" en lugar de "gcc".

Paso 5: Crear y configurar un archivo launch.json
Sigue las instrucciones de la sección 4.1, Paso 5 para crear y configurar el archivo launch.json. La configuración debería ser la misma que para C.

Paso 6: Verificar la configuración y probar el entorno de desarrollo
Para probar tu entorno de desarrollo de C++, sigue los pasos de la sección 4.1, Paso 6, pero crea un archivo de código fuente en C++ (con extensión .cpp) y escribe un programa simple en C++ en lugar de C.

Con estos pasos, tendrás un entorno de desarrollo completamente configurado para C++ en Visual Studio Code. Ahora puedes comenzar a desarrollar tus proyectos en C++ y aprovechar las herramientas y características de Visual Studio Code.

4.3. Configuración para Java

Para trabajar con Java en Visual Studio Code, es necesario configurar el entorno de desarrollo con la instalación de OpenJDK y extensiones específicas para Java. A continuación, se detalla una guía paso a paso para la configuración de VSCode para trabajar con Java:

Instalar OpenJDK:

Para compilar y ejecutar aplicaciones Java, necesitarás tener instalado el JDK (Java Development Kit) en tu computadora. En la elaboración de este documento (hoy) te recomiendo instalar OpenJDK, una implementación de código abierto de JDK.

a. Visita la página oficial de OpenJDK: https://adoptium.net
b. Selecciona la versión adecuada de OpenJDK para tu sistema operativo (Windows, macOS o Linux).
c. Descarga e instala OpenJDK siguiendo las instrucciones en pantalla.

Configurar la variable de entorno JAVA_HOME:

Es importante configurar la variable de entorno JAVA_HOME para que VSCode pueda ubicar el JDK.

a. Localiza la ruta de instalación de OpenJDK en tu computadora.
b. Configura la variable de entorno JAVA_HOME para apuntar a la ruta de instalación de OpenJDK.
c. Reinicia VSCode para que tome en cuenta los cambios en las variables de entorno.

Instalar extensiones para Java en VSCode:

VSCode admite el desarrollo de Java a través de extensiones específicas. Para instalar las extensiones necesarias… esto ya lo sabes y las extensiones de JAVA las veremos páginas adelante.

4.3. Configuración para Python

Para trabajar con Python en Visual Studio Code, es necesario configurar adecuadamente el entorno de desarrollo. Esto incluye la instalación de Python, la configuración del entorno en VSCode y la instalación de las extensiones necesarias para facilitar el desarrollo de aplicaciones en Python. A continuación, se presenta una guía paso a paso para configurar VSCode para trabajar con Python.

*Instalación de Python (*Descargar Python):

a. Visita la página oficial de Python en https://www.python.org/downloads/ para descargar la versión más reciente de Python compatible con tu sistema operativo (Windows, macOS o Linux).
b. Asegúrate de seleccionar la opción para agregar Python al PATH durante el proceso de instalación. Esto facilitará la ejecución de scripts Python desde la línea de comandos.

Configurar la variable de entorno PYTHONPATH (Opcional):

a. Aunque no es estrictamente necesario, configurar la variable de entorno PYTHONPATH puede ser útil para especificar directorios adicionales donde Python debe buscar módulos importados.
b. Puedes configurar PYTHONPATH desde el panel de control de tu sistema operativo, en la sección de variables de entorno.

Instalar extensiones para Python en VSCode:

a. Abre Visual Studio Code y accede al Marketplace de extensiones buscando "Python" o mediante el atajo Ctrl+Shift+X (o Cmd+Shift+X en macOS).
b. Instala la extensión oficial de Python desarrollada por Microsoft. Esta extensión proporciona soporte para el lenguaje Python, incluyendo características como IntelliSense, depuración, resaltado de sintaxis, snippets de código, y mucho más.

Configurar el intérprete de Python:

a. Al abrir un archivo Python en VSCode por primera vez, es posible que se te solicite seleccionar un intérprete de Python. Puedes hacerlo desde la barra de estado en la parte inferior de la ventana de VSCode o mediante el comando Python: Select Interpreter en la paleta de comandos (Ctrl+Shift+P o Cmd+Shift+P).
b. Selecciona el intérprete de Python que corresponda a la instalación que realizaste previamente.

Instalar y gestionar paquetes Python:

VSCode, a través de la extensión de Python, facilita la gestión de paquetes Python utilizando pip. Puedes instalar nuevos paquetes ejecutando comandos de pip en la terminal integrada de VSCode (Ctrl+ o Cmd+).

Configuración final y verificación:

a. Para verificar que tu entorno Python está configurado correctamente en VSCode, crea un nuevo archivo con la extensión .py y escribe un programa simple, como print("Hola, mundo en Python!").
b. Ejecuta el programa utilizando el botón de ejecución (▶) en la parte superior derecha de la ventana del editor o desde la terminal integrada.

Con estos pasos, habrás configurado con éxito Visual Studio Code para desarrollar aplicaciones en Python. VSCode, con su amplia gama de extensiones y herramientas de desarrollo, ofrece un entorno poderoso y flexible para el desarrollo en Python, desde scripts simples hasta proyectos complejos.

Extensiones y complementos recomendados

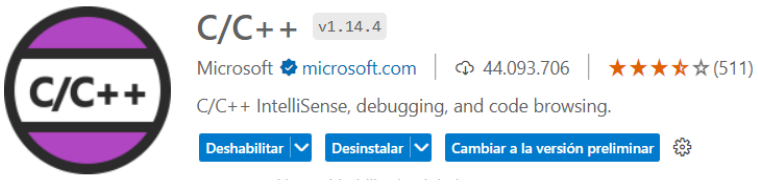
A continuación, veremos una selección personal de extensiones y complementos recomendados que pueden mejorar tu experiencia de aprendizaje y productividad en Visual Studio Code. Estas extensiones están diseñadas para facilitar el desarrollo de código en los lenguajes C, C++ y Java, así como para proporcionar herramientas adicionales que pueden ayudarte a mejorar tus habilidades y eficiencia en la programación. Esta selección se ha realizado considerando un enfoque práctico y orientado al aprendizaje, enfocado en las necesidades específicas de los estudiantes y futuros docentes de informática.

Algunas de estas extensiones son específicas para cada lenguaje, mientras que otras son más generales y pueden ser útiles independientemente del lenguaje que estés utilizando. A continuación, se presentan las extensiones recomendadas divididas en tres categorías: extensiones para C y C++, extensiones para Java y extensiones para productividad y aprendizaje.

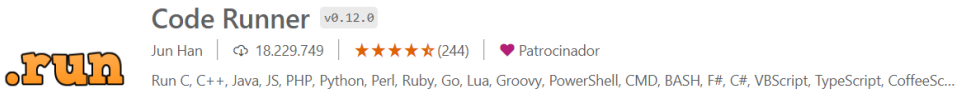
5.1. Extensiones para C y C++

Algunas de las extensiones más significativas y útiles para la programación en C y C++ en Visual Studio Code son:

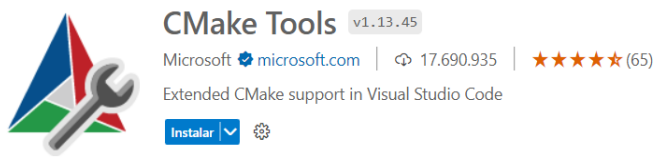
- C/C++ (Microsoft)
- Code Runner (Jun Han)
- CMake Tools (Microsoft)
- Doxygen Documentation Generator (Christoph Schlosser)



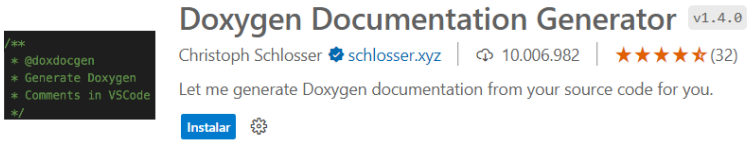
La extensión C/C++ desarrollada por Microsoft es fundamental, ya que proporciona características esenciales como el resaltado de sintaxis, la navegación por el código y la depuración. Además, es la base para la configuración del entorno de desarrollo que hemos discutido anteriormente en la sección 4.1 y 4.2.



Code Runner, desarrollado por Jun Han, es una extensión que permite ejecutar fácilmente fragmentos de código en varios lenguajes, incluidos C y C++. Esta herramienta es especialmente útil para probar rápidamente soluciones a problemas o experimentar con pequeños ejemplos de código sin tener que compilar y ejecutar todo el proyecto.



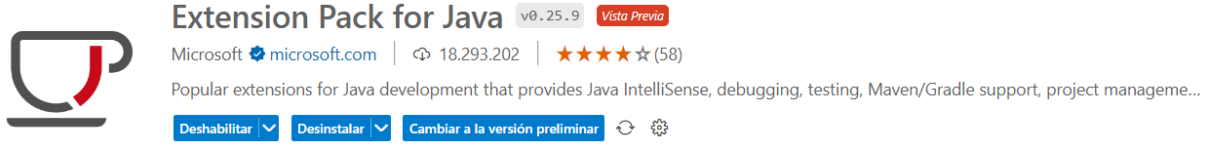
CMake Tools, de Microsoft, facilita el trabajo con proyectos basados en CMake, un sistema de construcción ampliamente utilizado en proyectos de C y C++. Esta extensión proporciona una interfaz intuitiva para configurar, construir y depurar proyectos CMake directamente desde Visual Studio Code.



Por último, Doxygen Documentation Generator, de Christoph Schlosser, simplifica la creación de documentación para proyectos de C y C++ utilizando el sistema de documentación Doxygen. Esta extensión facilita la generación y visualización de documentación en línea y en tiempo real, lo que puede ser útil para comprender mejor el código y colaborar con otros desarrolladores o estudiantes.

5.2. Extensiones para Java

Para comenzar, te recomiendo que instales la extensión Extension Pack for Java. Esta extensión incluye varias herramientas útiles que facilitan el desarrollo en Java, como soporte para el lenguaje, depuración, pruebas unitarias y más.



Solo con la instalación del Extension Pack for Java, obtendrás las siguientes extensiones:
Language Support for Java (TM) by Red Hat:
Proporciona soporte para Java, como linting, IntelliSense, formateo, refactorización y compatibilidad con Maven/Gradle, lo que facilita la escritura y mantenimiento del código Java.

Debugger for Java (Microsoft):
Se trata de un depurador ligero para Java en Visual Studio Code, que te permite depurar aplicaciones Java, establecer puntos de interrupción, inspeccionar variables y seguir la ejecución del programa.

Test Runner for Java (Microsoft):
Esta extensión permite ejecutar y depurar casos de prueba JUnit o TestNG en tu entorno de desarrollo, facilitando la realización de pruebas unitarias para tus aplicaciones Java.

Maven for Java (Microsoft):
Con Maven for Java, podrás administrar proyectos Maven, ejecutar objetivos, generar proyectos a partir de arquetipos y mejorar la experiencia del usuario para desarrolladores Java.

Project Manager for Java (Microsoft):
Esta extensión te ayudará a administrar proyectos Java en Visual Studio Code, ofreciendo una mejor organización y navegación en tu entorno de desarrollo.

IntelliCode (Microsoft):
IntelliCode es una herramienta de desarrollo asistida por inteligencia artificial que te proporciona sugerencias y autocompletado.



5.3. Extensiones para Python

Al trabajar con Python en Visual Studio Code, la instalación del Python Extension Pack es altamente recomendada. Este paquete incluye una colección de extensiones populares y favoritas para desarrollo en Python, proporcionando una amplia gama de funcionalidades desde soporte básico del lenguaje hasta herramientas avanzadas de depuración y gestión de entornos.



Extensiones Incluidas en el Python Extension Pack:
Python (Microsoft): Ofrece linting, depuración (multi-hilo, remota), IntelliSense, formateo de código, refactorización, pruebas unitarias, snippets, y más para Python. Incluye soporte para Data Science con Jupyter y PySpark.

Jinja: Añade soporte para el lenguaje de plantillas Jinja en Visual Studio Code, ideal para proyectos web que utilicen esta tecnología.

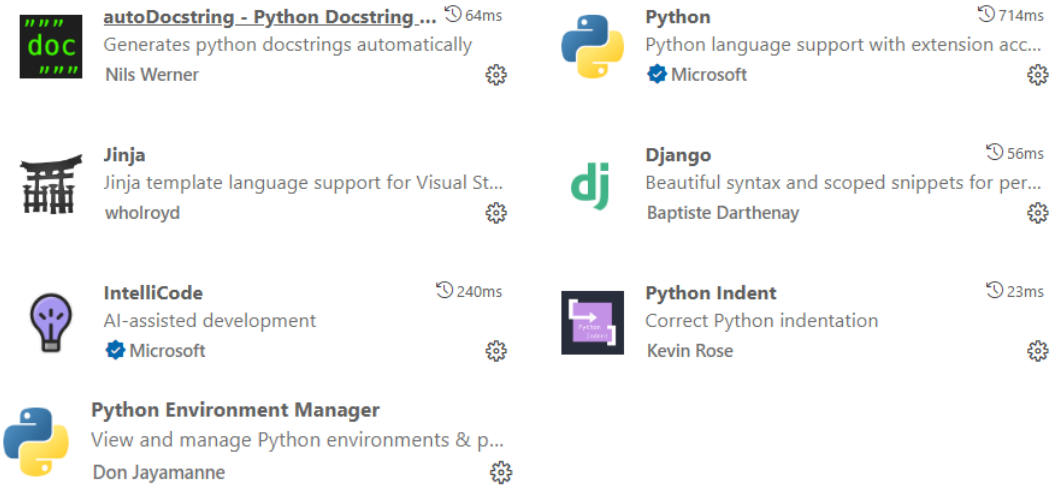
Django: Proporciona una sintaxis destacada y snippets específicos para Django, perfecto para desarrolladores trabajando bajo plazos ajustados.

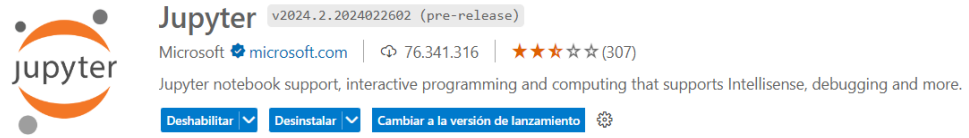
Visual Studio IntelliCode: Esta extensión utiliza la IA para ofrecer características de productividad para desarrolladores Python en Visual Studio Code, brindando sugerencias basadas en el entendimiento de tu código y el aprendizaje automático.

Python Environment Manager: Permite ver y gestionar todos tus entornos y paquetes Python desde un solo lugar, facilitando la gestión de dependencias y la configuración de proyectos.

Python Docstring Generator: Automatiza la inserción de bloques de comentarios Python, generando docstrings para clases y métodos basados en múltiples patrones de plantilla seleccionables.

Python Indent: Corrige la indentación en Python automáticamente, lo que es esencial para un lenguaje que depende de la estructura de indentación para definir el flujo del programa.





Aunque el Python Extension Pack enriquece significativamente tu entorno de desarrollo en Python con Visual Studio Code, una extensión que merece una mención especial y recomendación es Jupyter. Fundamental para aquellos en ciencia de datos, computación científica, y aprendizaje automático, Jupyter añade capacidades interactivas y de análisis directamente en Visual Studio Code, permitiendo una integración perfecta de los Notebooks de Jupyter en tu flujo de trabajo de desarrollo.

Por qué Instalar Jupyter para Visual Studio Code

Interactividad: Ejecuta código por celdas y visualiza los resultados al instante, lo que es ideal para el análisis exploratorio de datos y la experimentación con código.

Visualización de Datos: Integración directa con bibliotecas de visualización como Matplotlib y Seaborn, permitiendo la creación de gráficos dentro de los notebooks.

Soporte Markdown: Combina explicaciones en Markdown con tu código, facilitando la creación de documentos que explican el proceso de análisis paso a paso.

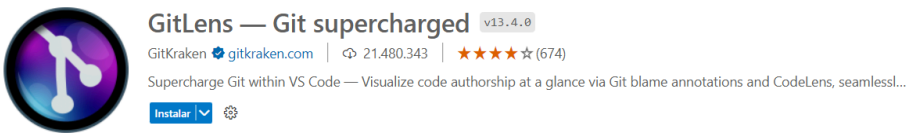
Compatibilidad: Trabaja con archivos .ipynb directamente en VSCode, manteniendo la compatibilidad con el ecosistema de Jupyter.

5.4 Extensiones para productividad y aprendizaje

Además de las extensiones específicas para los lenguajes C, C++ y Java, también hay varias extensiones en Visual Studio Code que pueden mejorar la productividad y el aprendizaje en general. Estas extensiones ofrecen una variedad de funciones que pueden ayudarte a mejorar tus habilidades de programación, organizar tu trabajo y aumentar la eficiencia. Aquí hay una selección de extensiones útiles con un enfoque en la práctica y el aprendizaje:

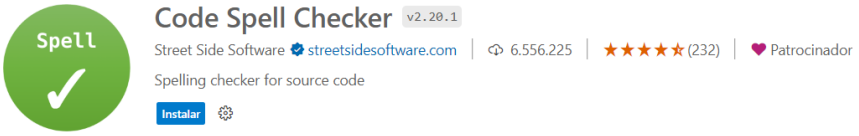


Live Share: Live Share, desarrollado por Microsoft, es una extensión que permite la colaboración en tiempo real en proyectos de Visual Studio Code. Puedes compartir tu espacio de trabajo con otros usuarios, editar código juntos, depurar en tiempo real y más. Es especialmente útil para estudiantes que trabajan en proyectos en grupo o para profesores que desean colaborar con sus alumnos.



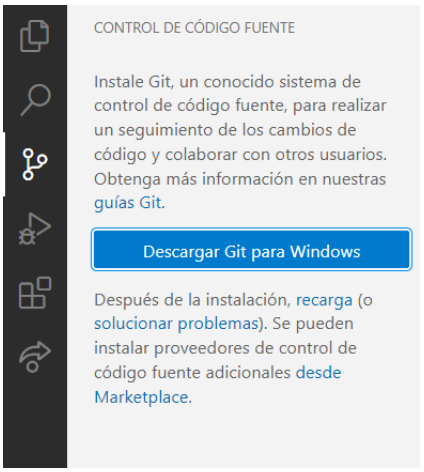
GitLens: GitLens, de Eric Amodio, mejora la integración de Git en Visual Studio Code, proporcionando funciones adicionales como la exploración de repositorios, la visualización de

historiales de cambios y la anotación de líneas de código con información relevante de Git. Esta extensión facilita el seguimiento y la colaboración en proyectos que utilizan el sistema de control de versiones Git.



Code Spell Checker: Code Spell Checker, de Street Side Software, es una extensión que verifica la ortografía en tus comentarios y cadenas de texto dentro de tu código. Esta herramienta puede ser útil para mantener un código limpio y fácil de leer, especialmente en proyectos compartidos con otros estudiantes o desarrolladores.

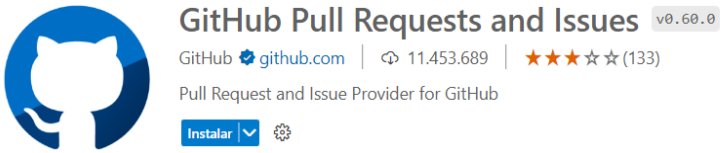
Integración con GitHub y control de versiones



Visual Studio Code ofrece una excelente integración con GitHub y otros sistemas de control de versiones. El control de versiones es una herramienta fundamental para cualquier desarrollador, ya que permite llevar un registro de los cambios realizados en el código, colaborar con otros desarrolladores y mantener un historial de las versiones del proyecto.

A continuación, se describen algunas extensiones y características de Visual Studio Code que facilitan la integración con GitHub y el control de versiones:

Git integrado: Visual Studio Code incluye soporte integrado para Git, lo que facilita el trabajo con repositorios y el seguimiento de los cambios en el código. Puedes realizar commits, crear y cambiar de ramas, resolver conflictos de merge y muchas otras acciones directamente desde el editor.



GitHub: Esta extensión oficial de Microsoft proporciona una integración aún más profunda con GitHub, permitiendo realizar acciones como clonar repositorios, crear y revisar pull requests, y navegar por los issues y repositorios directamente desde Visual Studio Code.

GitLens (opcional): Si deseas tener funcionalidades avanzadas de Git, GitLens es una excelente opción. Sin embargo, para estudiantes que recién comienzan con el control de versiones, la funcionalidad integrada de Git y la extensión de GitHub podrían ser suficientes. Puedes considerar instalar GitLens más adelante, cuando estés más familiarizado con Git y desees explorar características adicionales.

En general, el soporte integrado de Git y la extensión de GitHub deberían ser suficientes para la mayoría de los estudiantes y ayudar a mantener tus proyectos ordenados. Estas herramientas te permitirán llevar un registro de los cambios en tus proyectos y colaborar con otros desarrolladores de manera eficiente.

Personalización de VSCode: temas, atajos de teclado y ajustes

Personalizar Visual Studio Code puede mejorar significativamente tu experiencia de desarrollo y adaptar el entorno de trabajo a tus preferencias personales. A continuación, te presentamos algunas de las opciones de personalización más comunes en Visual Studio Code, incluidos temas, atajos de teclado y ajustes.

Temas
Los temas en Visual Studio Code te permiten cambiar la apariencia del editor, incluidos colores, fuentes y estilos. Hay una amplia variedad de temas disponibles en el Marketplace de Visual Studio Code, y también puedes crear tu propio tema personalizado.

Para instalar un tema, abre la pestaña de extensiones y busca el tema que deseas instalar. Una vez instalado, selecciona "Color Theme" en el menú "Preferences" (Preferencias) y elige el tema instalado en la lista. Puedes cambiar el tema en cualquier momento siguiendo estos pasos.

Atajos de teclado
Los atajos de teclado en Visual Studio Code te permiten realizar acciones rápidas y eficientes sin tener que navegar por los menús o hacer clic en los botones. Visual Studio Code viene con una amplia variedad de atajos de teclado predefinidos, y también puedes personalizarlos según tus preferencias.

Para ver y editar los atajos de teclado, selecciona "Keyboard Shortcuts" (Atajos de teclado) en el menú "Preferences" (Preferencias). Esto abrirá un editor donde puedes buscar atajos existentes y modificarlos o agregar nuevos atajos personalizados.

Ajustes
Visual Studio Code ofrece una gran cantidad de ajustes y opciones de configuración que te permiten personalizar el comportamiento y la apariencia del editor. Para acceder a los ajustes, selecciona "Settings" (Configuración) en el menú "Preferences" (Preferencias).

Los ajustes se pueden editar en formato JSON o a través de la interfaz gráfica. Algunos ejemplos de ajustes personalizables incluyen:

- Configuración del linter y formateador de código.
- Opciones de autocompletado y sugerencias de código.
- Configuración de la fuente y el espaciado.
- Comportamiento de la terminal integrada.
- Opciones de visualización y navegación en el editor.

Personalizar Visual Studio Code según tus preferencias puede mejorar tu experiencia de desarrollo y hacer que el entorno de trabajo sea más cómodo y eficiente. No dudes en experimentar con diferentes temas, atajos de teclado y ajustes para encontrar la configuración que mejor se adapte a tus necesidades y estilo de trabajo.

Consejos para sacar el máximo provecho de VSCode en el aprendizaje de programación

Para sacar el máximo provecho de Visual Studio Code en el aprendizaje de la programación, sigue estos consejos:

Familiarízate con los atajos de teclado: Conocer y utilizar los atajos de teclado en VSCode puede mejorar significativamente tu productividad y velocidad de trabajo. Puedes encontrar una lista completa de los atajos de teclado en la documentación oficial de VSCode o personalizarlos según tus preferencias.

Utiliza la función "Go to Definition": Esta función te permite navegar rápidamente al lugar donde una variable, función o clase está definida en tu código. Es especialmente útil cuando trabajas con proyectos más grandes o cuando estudias código escrito por otros.

Configura tu entorno de desarrollo: Asegúrate de tener las extensiones y herramientas necesarias para cada lenguaje de programación que estés aprendiendo. Dedica tiempo a personalizar tu entorno de desarrollo para que se ajuste a tus necesidades y preferencias.

Practica con ejercicios y proyectos: La mejor manera de aprender a programar es practicar con ejercicios y proyectos reales. Utiliza VSCode para crear proyectos y experimentar con diferentes características del lenguaje de programación que estás aprendiendo.

Aprovecha la comunidad: La comunidad de VSCode es activa y siempre dispuesta a ayudar. Participa en foros, grupos de discusión y redes sociales relacionadas con VSCode y los lenguajes de programación que estás aprendiendo. No dudes en hacer preguntas y compartir tus conocimientos con otros.

Utiliza la función de depuración: Aprende a utilizar la depuración integrada en VSCode para detectar y solucionar errores en tu código. Esta herramienta es invaluable para mejorar la calidad de tu código y tu capacidad para solucionar problemas.

Consulta la documentación: La documentación oficial de VSCode es un recurso valioso para aprender a utilizar y personalizar el editor. Consulta la documentación con regularidad para estar al tanto de las últimas funciones y características.

Mantén tu VSCode actualizado: Asegúrate de actualizar regularmente VSCode y sus extensiones para aprovechar las últimas mejoras y correcciones de errores. Esto garantiza que estés utilizando la versión más estable y funcional del editor.

Al seguir estos consejos, podrás aprovechar al máximo las características y herramientas que Visual Studio Code tiene para ofrecer y mejorar tus habilidades de programación en C, C++ y Java.

Conclusión

A lo largo de este documento, hemos explorado cómo configurar y personalizar Visual Studio Code para un aprendizaje eficiente de la programación en lenguajes como Python, C, C++ y Java. Hemos discutido la importancia de contar con un entorno de desarrollo adecuado, las extensiones y complementos recomendados, así como los recursos y herramientas adicionales para mejorar la experiencia de aprendizaje.

Uruguay está atravesando una transformación educativa enfocada en el desarrollo de competencias y habilidades que preparen a los estudiantes para los desafíos del siglo XXI. La programación y la alfabetización digital son componentes clave de este cambio, y herramientas como Visual Studio Code pueden desempeñar un papel fundamental en el proceso.

Además, el avance de la inteligencia artificial (IA) y su creciente integración en el mundo de la programación, como se evidencia en herramientas como GitHub Copilot, brinda nuevas oportunidades para mejorar la enseñanza y el aprendizaje de la programación. Estas tecnologías pueden ofrecer apoyo personalizado a los estudiantes y facilitar el desarrollo de habilidades de resolución de problemas y pensamiento crítico.

En este contexto, es esencial que educadores, estudiantes y profesionales de la tecnología colaboren y aprovechen al máximo las herramientas y recursos disponibles. Visual Studio Code es una excelente plataforma para explorar y dominar los lenguajes de programación, y su uso en la educación puede potenciar la formación de futuros profesionales capaces de enfrentar los desafíos tecnológicos y sociales que se avecinan.